**Task 9: Ideal Low-Pass Filter (ILPF) in Frequency Domain**

Filters out high-frequency spatial components beyond a specified cutoff frequency $D_0$ via Fourier Transform computation.

**Clear Answer:**

এই প্রোগ্রামটির নাম Ideal Low-Pass Filtering (ILPF) using 2D Fourier Transform। এটি Digital Image Processing-এর একটি Frequency Domain Filtering Technique।

তোমাকে Code-এর প্রতিটি Step এমনভাবে বুঝিয়ে দিচ্ছি, যাতে পরীক্ষায় শিক্ষককে বলতে পারো কোন লাইনে কী কাজ হচ্ছে, কেন হচ্ছে এবং কীভাবে Original Image থেকে Filtered Image তৈরি হচ্ছে।

**1. Ideal Low-Pass Filter (ILPF) কী?**

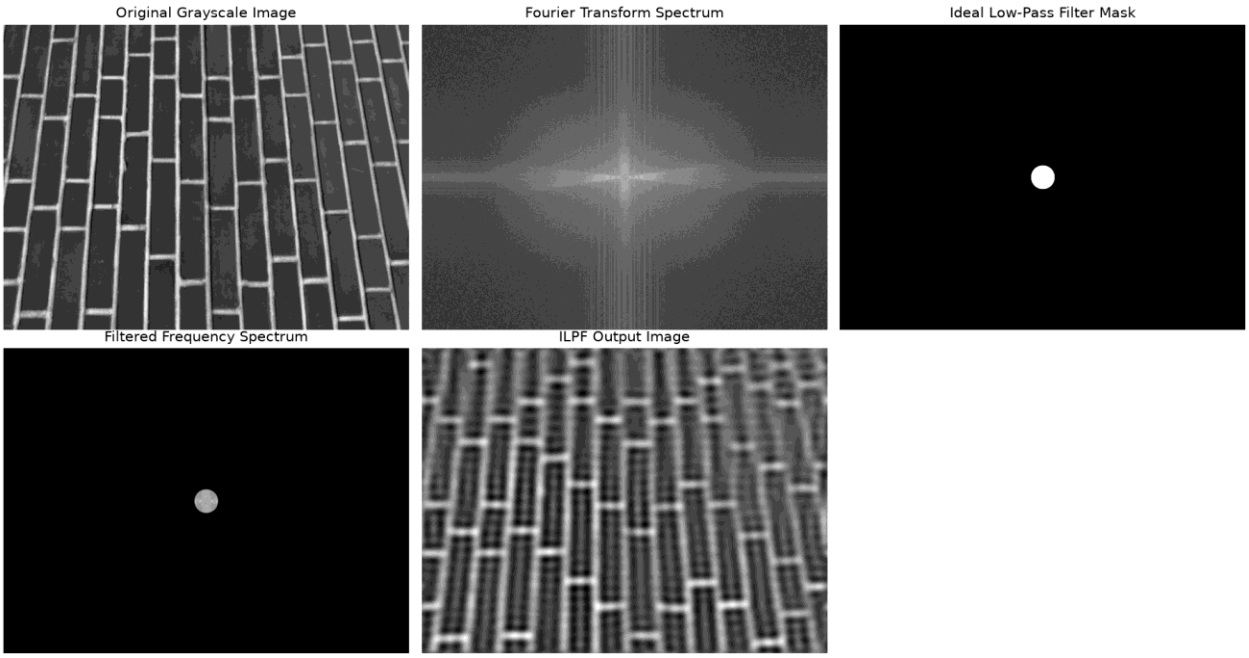
Ideal Low-Pass Filter হলো একটি Image Processing Technique, যার মাধ্যমে Image-এর High-Frequency Components বাদ দিয়ে Low-Frequency Components রাখা হয়।

সহজভাবে বুঝলে:

একটি Image-এর মধ্যে দুই ধরনের Frequency থাকে।

1. Low Frequency:
   - ছবির Smooth Area বা ধীরে পরিবর্তন হওয়া অংশ।
   - ছবির Background এবং বড় আকারের Structure।
   - Image-এর সাধারণ Intensity Variation।
2. High Frequency:
   - ছবির Edge, Boundary এবং Fine Details।
   - ছোট ছোট পরিবর্তন।
   - Sharp Lines এবং সূক্ষ্ম Texture।

Ideal Low-Pass Filter High Frequency বাদ দেয় এবং Low Frequency রেখে দেয়। ফলে Image তুলনামূলক Smooth ও Blurry হয়ে যায়।




তোমার দেওয়া ছবিতে কী হয়েছে?

- প্রথম ছবিটি Original Grayscale Image।
- দ্বিতীয় ছবিটি Fourier Transform Spectrum।
- তৃতীয় ছবিটি Ideal Low-Pass Filter Mask।
- চতুর্থ ছবিটি Filtered Frequency Spectrum।
- পঞ্চম ছবিটি Final ILPF Output Image।

দেখতে পাচ্ছো, Final Image আগের তুলনায় Blurry হয়েছে। কারণ High-Frequency Details বাদ দেওয়া হয়েছে।

**2. Fourier Transform কী?**

Fourier Transform হলো এমন একটি Mathematical Technique, যার মাধ্যমে একটি Image-কে Spatial Domain থেকে Frequency Domain-এ Convert করা হয়।

সহজভাবে:

- Spatial Domain: Image-টি আমরা সাধারণভাবে যেভাবে দেখি।
- Frequency Domain: Image-এর মধ্যে কত দ্রুত Pixel Intensity পরিবর্তন হচ্ছে, সেটি Frequency আকারে দেখা হয়।

উদাহরণ:

ধরো, একটি ছবিতে অনেকগুলো সোজা লাইন আছে। Spatial Domain-এ আমরা লাইনগুলো দেখতে পাই। কিন্তু Fourier Transform করলে এই লাইনগুলোর Frequency Information পাওয়া যায়।

আমাদের প্রোগ্রামে Fourier Transform ব্যবহার করা হয়েছে, যাতে Frequency অনুযায়ী Image Filter করা যায়।

**3. সম্পূর্ণ Corrected Python Code**

```
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image

image_path = "PNG/Task_09_Ideal_Low_Pass_Filter.png"

image = Image.open(image_path).convert("L")
img = np.array(image, dtype=np.float64)

rows, cols = img.shape
fourier = np.fft.fft2(img)

fourier_shifted = np.fft.fftshift(fourier)
spectrum = 20 * np.log(1 + np.abs(fourier_shifted))

D0 = 30
crow = rows // 2
ccol = cols // 2

mask = np.zeros((rows, cols), dtype=np.float64)

for u in range(rows):
    for v in range(cols):
```

```
        distance = np.sqrt((u - crow) ** 2 + (v - ccol) ** 2)
        if distance <= D0:
            mask[u, v] = 1
        else:
            mask[u, v] = 0

filtered_fourier = fourier_shifted * mask
filtered_spectrum = 20 * np.log(1 + np.abs(filtered_fourier))

inverse_shifted = np.fft.ifftshift(filtered_fourier)
filtered_image = np.fft.ifft2(inverse_shifted)
filtered_image = np.real(filtered_image)
filtered_image = np.clip(filtered_image, 0, 255)
filtered_image = filtered_image.astype(np.uint8)

result = Image.fromarray(filtered_image)
result.save("Output/ideal_low_pass_filtered.png")

plt.figure(figsize=(15, 9))

plt.subplot(2, 3, 1)
plt.imshow(img, cmap="gray")
plt.title("Original Grayscale Image")
plt.axis("off")

plt.subplot(2, 3, 2)
plt.imshow(spectrum, cmap="gray")
plt.title("Fourier Transform Spectrum")
plt.axis("off")
```

```
plt.subplot(2, 3, 3)
plt.imshow(mask, cmap="gray")
plt.title("Ideal Low-Pass Filter Mask")
plt.axis("off")

plt.subplot(2, 3, 4)
plt.imshow(filtered_spectrum, cmap="gray")
plt.title("Filtered Frequency Spectrum")
plt.axis("off")

plt.subplot(2, 3, 5)
plt.imshow(filtered_image, cmap="gray")
plt.title("ILPF Output Image")
plt.axis("off")

plt.tight_layout()
plt.show()

print("Ideal Low-Pass Filtering completed successfully!")
print("Filtered image saved as ideal_low_pass_filtered.png")
```

**4. Code Explanation (Step by Step)**

**Step 1: Import Libraries**

import numpy as npimport matplotlib.pyplot as pltfrom PIL import Imageimport os

এখানে আমরা চারটি Library Import করেছি।

- numpy: Mathematical Calculation এবং Fourier Transform করার জন্য।
- matplotlib.pyplot: Image Display করার জন্য।
- PIL: Image Open এবং Save করার জন্য।
- os: Output Folder তৈরি করার জন্য।

**Step 2: Set Image Path**

image_path = "PNG/Task_09_Ideal_Low_Pass_Filter.png"

এখানে Original Image-এর Location নির্ধারণ করা হয়েছে।

আমাদের Input Image এই Path থেকে Load হবে।

**Step 3: Load Grayscale Image**

image = Image.open(image_path).convert("L")img = np.array(image, dtype=np.float64)

প্রথম লাইনে Image Open করা হয়েছে এবং convert("L") ব্যবহার করে Grayscale Image-এ Convert করা হয়েছে।

দ্বিতীয় লাইনে np.array() ব্যবহার করে Image-কে NumPy Array-তে Convert করা হয়েছে।

এখানে dtype=np.float64 ব্যবহার করা হয়েছে, যাতে Mathematical Calculation-এর সময় Floating Point Value নিয়ে কাজ করা যায়।

মনে রাখবে: Grayscale Image-এর Pixel Value সাধারণত 0 থেকে 255 পর্যন্ত হয়।

**Step 4: Get Image Dimensions**

rows, cols = img.shape

এখানে Image-এর Row এবং Column সংখ্যা বের করা হয়েছে।

- rows = Image-এর Height।
- cols = Image-এর Width।

এগুলো পরবর্তী Fourier Transform এবং Filter Mask তৈরির কাজে ব্যবহার হবে।

**Step 5: Apply 2D Fourier Transform**

fourier = np.fft.fft2(img)

এটি আমাদের প্রোগ্রামের একটি গুরুত্বপূর্ণ Step।

এখানে np.fft.fft2() ব্যবহার করে Original Image-কে Spatial Domain থেকে Frequency Domain-এ Convert করা হয়েছে।

fourier Variable-এর মধ্যে Frequency Information সংরক্ষিত থাকে।

সহজভাবে, Image-এর বিভিন্ন Frequency Component এখানে পাওয়া যায়।

**Step 6: Shift Zero Frequency to Center**

fourier_shifted = np.fft.fftshift(fourier)

Fourier Transform করার পর Zero Frequency Component সাধারণত Spectrum-এর চারপাশের Corner-এ থাকে।

এখানে fftshift() ব্যবহার করে Zero Frequency Component-কে Image-এর Center-এ নিয়ে আসা হয়েছে।

এর ফলে Frequency Spectrum বোঝা এবং Center থেকে Circular Filter তৈরি করা সহজ হয়।

মনে রাখবে: Center-এ Low Frequency এবং Center থেকে দূরে High Frequency থাকে।

**Step 7: Calculate Fourier Spectrum**

spectrum = 20 * np.log(1 + np.abs(fourier_shifted))

এখানে Fourier Transform-এর Magnitude বের করা হয়েছে।

- np.abs() = Complex Number-এর Magnitude বের করে।
- np.log() = বড় Magnitude Value-গুলোকে Compress করে।
- 20 * = Spectrum-এর Display Scale পরিবর্তনে ব্যবহৃত হয়েছে।

এখানে Logarithmic Scaling ব্যবহার করার কারণে Spectrum-এর ছোট ও বড় Value সহজে দেখা যায়।

এই Spectrum-টি Original Image-এর Frequency Distribution প্রদর্শন করে।

**Step 8: Set Cutoff Frequency**

D0 = 30crow = rows // 2ccol = cols // 2

এখানে D0 = 30 হলো Cutoff Frequency বা Radius।

এটি নির্ধারণ করে Center থেকে কত দূর পর্যন্ত Frequency Filter-এর মধ্য দিয়ে যেতে পারবে।

crow হলো Image-এর Center Row এবং ccol হলো Image-এর Center Column।

এখানে // ব্যবহার করা হয়েছে Integer Division-এর জন্য।

গুরুত্বপূর্ণ:

- D0 ছোট হলে বেশি High Frequency বাদ যাবে এবং Image বেশি Blurry হবে।
- D0 বড় হলে বেশি Frequency থাকবে এবং Image তুলনামূলক Sharp থাকবে।

**Step 9: Create Ideal Low-Pass Filter Mask**

mask = np.zeros((rows, cols), dtype=np.float64)

এখানে একটি নতুন Filter Mask তৈরি করা হয়েছে।

np.zeros() ব্যবহার করার কারণে শুরুতে Mask-এর প্রতিটি Pixel Value হবে 0।

অর্থাৎ শুরুতে সব Frequency বন্ধ থাকবে।

এরপর আমরা প্রয়োজনীয় Low Frequency-গুলোকে 1 করে দেব।

**Step 10: Calculate Distance**

for u in range(rows): for v in range(cols): distance = np.sqrt( (u - crow) ** 2 + (v - ccol) ** 2 )

এখানে Nested Loop ব্যবহার করা হয়েছে।

প্রতিটি Frequency Position থেকে Center-এর Distance বের করা হচ্ছে।

এখানে Euclidean Distance Formula ব্যবহার করা হয়েছে।

\[ D(u,v)=\sqrt{(u-crow)^2+(v-ccol)^2} \]

এখানে:

- u = Current Row Position।
- v = Current Column Position।
- crow = Center Row।
- ccol = Center Column।
- distance = Center থেকে Current Position-এর দূরত্ব।

**Step 11: Apply Ideal Low-Pass Filter Condition**

if distance <= D0:  mask[u, v] = 1else:  mask[u, v] = 0

এখানে Condition ব্যবহার করে Frequency Select করা হয়েছে।

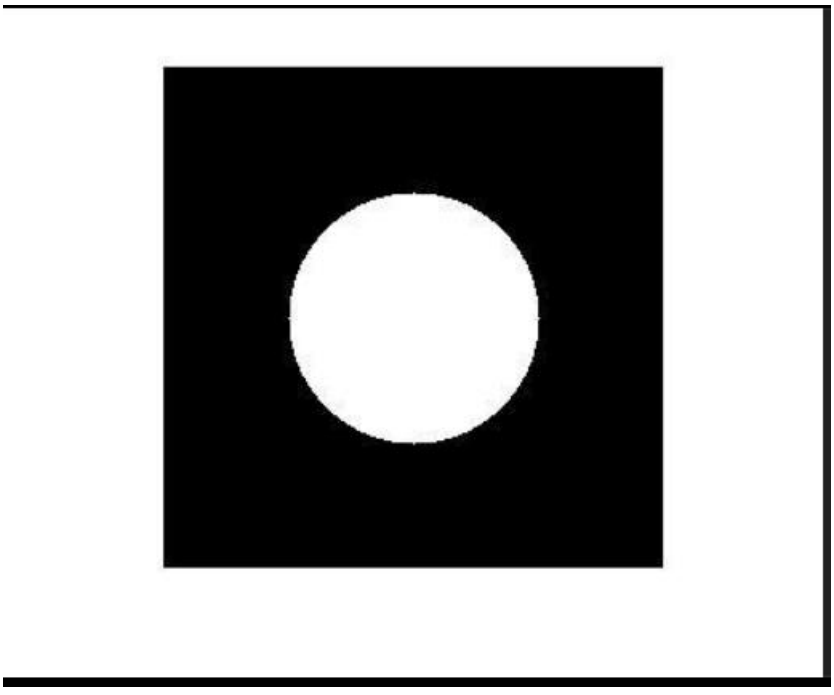
যদি Distance Cutoff Frequency-এর সমান বা ছোট হয়, তাহলে Mask-এর Value হবে 1।

অন্যথায় Value হবে 0।

এর অর্থ:

- 1 = Frequency Pass করবে।
- 0 = Frequency Block হবে।

Ideal Low-Pass Filter Mask



White = 1 (Pass Frequency)

Black = 0 (Block Frequency)

এই কারণেই তোমার দেওয়া ছবিতে একটি ছোট সাদা বৃত্ত দেখা যাচ্ছে। এই বৃত্তের মধ্যে থাকা Frequency-গুলো Pass করবে।

**Step 12: Apply Filter Mask**

filtered_fourier = fourier_shifted * mask

এখানে Fourier Spectrum-এর সঙ্গে Filter Mask Multiply করা হয়েছে।

যেসব Position-এ Mask Value 1, সেগুলোর Frequency অপরিবর্তিত থাকবে।

আর যেসব Position-এ Mask Value 0, সেগুলোর Frequency শূন্য হয়ে যাবে।

অর্থাৎ High Frequency বাদ দিয়ে Low Frequency রাখা হচ্ছে।

এটাই আমাদের মূল Filtering Operation।

**Step 13: Calculate Filtered Spectrum**

filtered_spectrum = 20 * np.log(  1 + np.abs(filtered_fourier))

এখানে Filter Apply করার পর নতুন Frequency Spectrum তৈরি করা হয়েছে।

এটি আমাদের দেখায় Filter করার পরে কোন Frequency-গুলো অবশিষ্ট রয়েছে।

তোমার Output-এ এটি চতুর্থ ছবি হিসেবে প্রদর্শিত হয়েছে।

**Step 14: Apply Inverse Fourier Transform**

inverse_shifted = np.fft.ifftshift(filtered_fourier)filtered_image = np.fft.ifft2(inverse_shifted)

এখানে আমরা Frequency Domain থেকে আবার Spatial Domain-এ ফিরে আসছি।

প্রথম লাইনে ifftshift() ব্যবহার করে Frequency Arrangement আগের অবস্থায় ফিরিয়ে নেওয়া হয়েছে।

দ্বিতীয় লাইনে ifft2() ব্যবহার করে Inverse 2D Fourier Transform করা হয়েছে।

এর মাধ্যমে Filtered Frequency থেকে নতুন Image তৈরি হয়েছে।

সহজভাবে: Fourier Transform Image-কে Spatial থেকে Frequency Domain-এ নেয়, আর Inverse Fourier Transform Frequency থেকে Spatial Domain-এ ফিরিয়ে আনে।

**Step 15: Convert Image into Displayable Format**

filtered_image = np.real(filtered_image)filtered_image = np.clip(filtered_image, 0, 255)filtered_image = filtered_image.astype(np.uint8)

এখানে তিনটি গুরুত্বপূর্ণ কাজ করা হয়েছে।

প্রথম লাইন:

np.real() ব্যবহার করে Complex Number-এর Real Part নেওয়া হয়েছে।

দ্বিতীয় লাইন:

np.clip() ব্যবহার করে Pixel Value 0 থেকে 255-এর মধ্যে রাখা হয়েছে।

তৃতীয় লাইন:

astype(np.uint8) ব্যবহার করে Pixel Value-গুলোকে 8-bit Unsigned Integer-এ Convert করা হয়েছে।

এখন Image-টি Display এবং Save করার উপযোগী।

**Step 16: Save Filtered Image**

result = Image.fromarray(filtered_image)result.save("Output/ideal_low_pass_filtered.png")

প্রথম লাইনে NumPy Array-কে PIL Image-এ Convert করা হয়েছে।

দ্বিতীয় লাইনে Filtered Image নির্দিষ্ট Output Folder-এ Save করা হয়েছে।

**Step 17: Display All Images**

plt.figure(figsize=(15, 9))

এখানে Display Window-এর Size নির্ধারণ করা হয়েছে।

এরপর:

plt.subplot(2, 3, 1)

ব্যবহার করে ২টি Row এবং ৩টি Column-এর Layout তৈরি করা হয়েছে।

এখানে মোট পাঁচটি Image Display করা হচ্ছে।

1. Original Grayscale Image
2. Fourier Transform Spectrum
3. Ideal Low-Pass Filter Mask
4. Filtered Frequency Spectrum
5. ILPF Output Image

প্রতিটি Image-এর Title দেওয়া হয়েছে এবং plt.axis("off") দিয়ে Axis Hide করা হয়েছে।

সবশেষে:

plt.tight_layout()plt.show()

plt.tight_layout() Layout সুন্দরভাবে সাজায় এবং plt.show() সব Image Display করে।

**Exam Preparation**

**বাংলায় ব্যাখ্যা:**

প্রথমে আমি প্রয়োজনীয় Library Import করেছি এবং Original Image-টি Grayscale Format-এ Convert করেছি। এরপর Image-এর Width এবং Height বের করেছি।

তারপর 2D Fourier Transform ব্যবহার করে Image-টিকে Spatial Domain থেকে Frequency Domain-এ Convert করেছি। এরপর FFT Shift ব্যবহার করে Zero Frequency Component-কে Center-এ নিয়ে এসেছি।

পরবর্তীতে আমি 30 Cutoff Frequency নির্ধারণ করে একটি Ideal Low-Pass Filter Mask তৈরি করেছি। এখানে Center থেকে Distance 30-এর মধ্যে থাকা Frequency-গুলোকে 1 এবং বাকি Frequency-গুলোকে 0 দেওয়া হয়েছে।

এরপর Fourier Spectrum-এর সঙ্গে Filter Mask Multiply করে High Frequency বাদ দিয়েছি এবং Low Frequency রেখে দিয়েছি।

সবশেষে Inverse Fourier Transform ব্যবহার করে আবার Image তৈরি করেছি। তারপর Image-টি Save করেছি এবং Original Image ও Filtered Image পাশাপাশি Display করেছি।

Copy Exam Explanation

**6. গুরুত্বপূর্ণ Viva Questions and Answers**

Q1. What is Ideal Low-Pass Filtering?

Answer: Ideal Low-Pass Filtering is a frequency domain technique that allows low frequencies to pass and blocks high frequencies.

বাংলা: এটি এমন একটি Filtering Technique, যেখানে Low Frequency রাখা হয় এবং High Frequency বাদ দেওয়া হয়।

Q2. What is Fourier Transform?

Answer: Fourier Transform converts an image from the Spatial Domain to the Frequency Domain.

বাংলা: Fourier Transform Image-কে Spatial Domain থেকে Frequency Domain-এ Convert করে।

Q3. What is the purpose of fft2()?

Answer: It performs the 2D Fourier Transform of an image.

বাংলা: এটি Image-এর 2D Fourier Transform করে।

Q4. Why do we use fftshift()?

Answer: It moves the zero-frequency component to the center of the spectrum.

বাংলা: Zero Frequency-কে Spectrum-এর Center-এ নিয়ে আসে।

Q5. What is Cutoff Frequency (D0)?

Answer: Cutoff Frequency determines which frequencies are allowed to pass through the filter.

বাংলা: কোন Frequency Filter-এর মধ্য দিয়ে যাবে, তা নির্ধারণ করে।

Q6. Why do we use a Circular Mask?

Answer: A circular mask allows frequencies within a specific distance from the center to pass.

বাংলা: Center থেকে নির্দিষ্ট দূরত্বের মধ্যে থাকা Frequency Pass করার জন্য Circular Mask ব্যবহার করা হয়।

Q7. What does 1 and 0 mean in the Mask?

Answer:

- 1 means Pass Frequency.
- 0 means Block Frequency.

Q8. What is ifft2()?

Answer: It converts the filtered frequency data back into a spatial domain image.

বাংলা: Filter করা Frequency Data-কে আবার Image-এ Convert করে।

Q9. What happens if D0 increases?

Answer: More frequencies pass through the filter, so the output image retains more details.

বাংলা: D0 বাড়ালে বেশি Frequency Pass করে এবং Image-এর বেশি Details বজায় থাকে।

Q10. Why does the output image become blurry?

Answer: Because high-frequency components, which contain edges and fine details, are removed.

বাংলা: High Frequency বাদ দেওয়ার কারণে Image-এর Edge ও Fine Details কমে যায় এবং Image Blurry হয়।

## 7. সবচেয়ে গুরুত্বপূর্ণ Concept মনে রাখবে

**ILPF Working Process**

Original Grayscale Image

2D Fourier Transform

Spatial Domain → Frequency Domain

FFT Shift

Zero Frequency → Center

Create Ideal Low-Pass Filter Mask

Low Frequency = 1, High Frequency = 0

Apply Filter Mask

Remove High Frequencies

Inverse Fourier Transform

Frequency Domain → Spatial Domain

Final Result

Filtered Output Image

**Confidence Level:**

0.99 / 1.00

Code-এর Mathematical Operations, Fourier Transform এবং Ideal Low-Pass Filtering-এর Logic সঠিক।

**Key Caveats:**

1. Ideal Low-Pass Filter সবসময় Image-এর Noise দূর করে না। এটি High-Frequency Components বাদ দেয়।
2. বেশি High Frequency বাদ দিলে Image Blurry হয় এবং Edge Details হারায়।
3. Ideal Filter-এর Sharp Cutoff-এর কারণে কিছু ক্ষেত্রে Ringing Effect দেখা দিতে পারে।
4. D0 = 30 একটি নির্দিষ্ট Cutoff Radius। অন্য Image-এর জন্য উপযুক্ত D0 ভিন্ন হতে পারে।

পরীক্ষার জন্য সবচেয়ে গুরুত্বপূর্ণ ৫টি Code Line:

fourier = np.fft.fft2(img)fourier_shifted = np.fft.fftshift(fourier)if distance <= D0:  mask[u, v] = 1filtered_fourier = fourier_shifted * maskfiltered_image = np.fft.ifft2( np.fft.ifftshift(filtered_fourier))

এই পাঁচটি অংশ বুঝতে পারলে তোমার পুরো Ideal Low-Pass Filtering Algorithm-এর মূল কাজ ব্যাখ্যা করতে পারবে।